

QATIPv3 – Azure Lab 6a

Pipelining with Terraform Cloud

Contents

Overview	1
Before you begin	1
Task1 Signing up for a free Terraform Cloud account.	1
Task2 Creating a Terraform Cloud Workspace	3
Task3 Integrate Terraform client and Terraform Cloud Workspace	8

Overview

This lab introduces Terraform Cloud and will demonstrate moving state storage and command execution off the local machine and into Terraform Cloud.

The solution to this lab is in folder C:\Labs\solutions\lab6. Try to use this only as a last resort if you are struggling to complete the step-by-step processes.

Before you begin

Before you begin, ensure you have followed the lab setup instructions for Azure and navigated to the lab6 folder in VSC terminal and explorer.

Task1 Signing up for a free Terraform Cloud account.

To successfully complete this lab, you will need an active email account. Consider creating a new burner email account that you can use purely for this lab.

1. Browse to <https://app.terraform.io/>..

HashiCorp Terraform Cloud

Sign in to Terraform Cloud

 Continue with HCP account

OR

Username or email

Password

[Forgot password?](#)

Sign in

Sign in with SSO.

Need to sign up? Create your free account.

[View Terraform Offerings to find out which one is right for you.](#)

2. Complete the enrolment form to create a new free account..

Create an account [Have an account? Sign in](#)

 Continue with HCP account

OR

Username

Email

Password

☐ I agree to the Terms of Use.

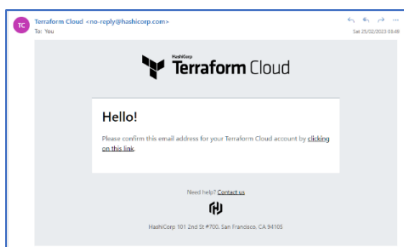
☐ I acknowledge the Privacy Policy.

[Please review the Terms of Use and Privacy Policy.](#)

Create account

3. A confirmation email will be sent to you.

4. Acknowledge the email sent...



5. A new browser session will open. You can safely close the first session tab. There are 3 setup workflow options. We will begin with **Start from scratch...**

Welcome to Terraform Cloud!

Choose your setup workflow

- Try an example configuration** Recommended for OSS users
Perform your first Terraform Cloud run using a sample configuration with the CLL.
[Learn More](#)
- Start from scratch**
Start with a blank slate. Best for users who are already familiar with Terraform Cloud.
[Learn More](#)
- Import local state**
Start with existing state. Best for users who already manage infrastructure with Terraform OSS.
[Learn More](#)

6. As this is a new account you are required to create a new organization. Chose an organization name and complete the form...

Create a new organization

Organizations are privately shared spaces for teams to collaborate on infrastructure. [Learn more](#) about organizations in Terraform Cloud.

Organization name
e.g. company-name

Organization names must be unique and will be part of your resource names used in various tools, for example `QATIP/aaa-prod`.

Email address

The organization email is used for any future notifications, such as billing alerts, and the organization avatar, via [gravatar.com](#).

[Create organization](#)

Task2 Creating a Terraform Cloud Workspace

1. Before creating our first workspace we must consider the credentials that Terraform Cloud will use as its authority to perform tasks in Azure on our behalf. We will create a service principal and store its security credentials as terraform variables in Terraform Cloud.
2. Switch to the IDE terminal and enter the following command as a single entry, inserting your subscription details as indicated...

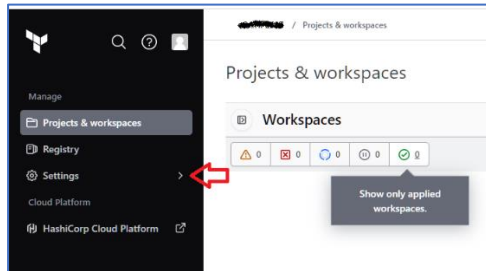
```
az ad sp create-for-rbac --name "myappsp" --role Contributor --scopes /subscriptions/insert your subscription here
```

3. Copy the output to a local text file for later use. It should be similar to the below, with your own unique values..

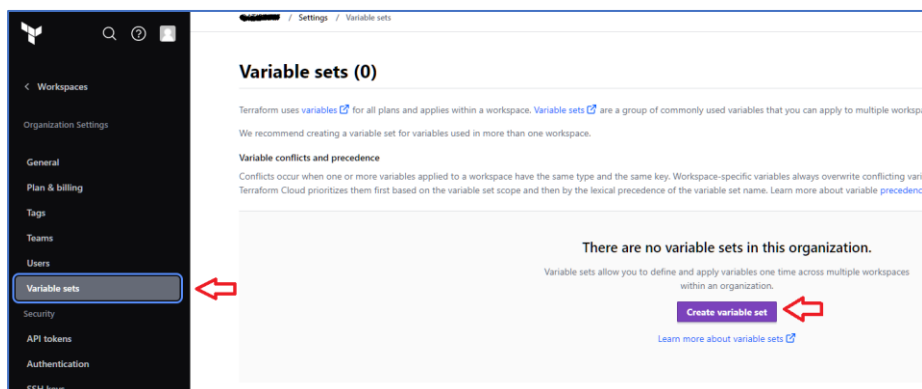
```
"appId": "e07200b0-8451-4f1a-a85a-44369e90870b",  
"displayName": "myappsp",
```

"password":
"ly58Q~YKrjNsA_h4vQXU1dRnhhOWNnO_emabWcYp",
"tenant": "e35b03b3-c8fe-4c25-a5b7-5cc0aa85d11b"

4. Switch back to Terraform Cloud. In the Projects and workspaces view, select `Settings`...



5. This will take us to the Organization menu. Select `Variable Sets` and then `Create variable set`...



6. Name the set **Cloud Credentials**, apply the set globally across all workspaces, and click to add your first variable...

Create a new variable set

Variable sets allow you to define and apply variables one time across multiple workspaces within an organization.

Configure settings

Name
Cloud Credentials

Description (Optional)

Variable set scope

☒ Apply globally
All current and future projects in this organization will access this variable set.

☐ Apply to specific projects and workspaces

Variables

You can add any number of Terraform and Environment variables. Terraform will use these variables for all plan and apply operations in the specified workspaces.

Key	Value
There are no variables added.	

+ Add variable

Create variable set

Select variable category

☒ **Terraform variable**
These variables should match the declarations in your configuration. Click the HCL box to use interpolation or set a non-string value.

☐ **Environment variable**
These variables are available in the Terraform runtime environment.

Key	Value
key	value

☐ HCL ☐ Sensitive

Description (Optional)
description (optional)

Add variable Cancel

7. Create 4 Terraform variables as shown below, marking **client_secret** as **Sensitive**

client_secret - populate with **password** collected earlier (no quotes)

client_id - populate with **app_id** collected earlier (no quotes)

subscription_id - populate with your **subscription**

tenant_id - populate with **tenant** collected earlier (no quotes)

Variables

You can add any number of Terraform and Environment variables. Terraform will use these variables for all plan and apply operations in the specified workspaces.

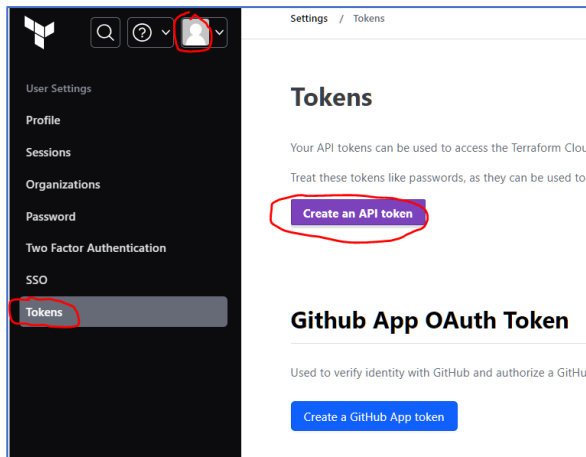
Key	Value	Category
cleint_secret SENSITIVE	Sensitive - write only	terraform
client_id	e07200b0-8451-4f1a-a85a-44369e90870b	terraform
subscription_id	911e740f-0030-41d9-839c-c3579ec74ee4	terraform
tenant_id	e35b03b3-c8fe-4c25-a5b7-5cc0a85d11b	terraform

+ Add variable

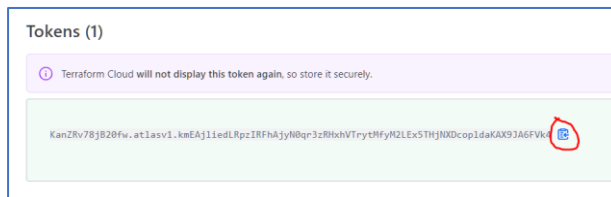
Save variable set Delete variable set

8. Save the variable set

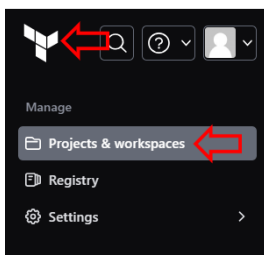
9. We also need to generate an api token within Terraform Cloud that will be used as authentication credentials from the IDE. Navigate to your **User Settings** and select **Tokens**..
10. Select **Create an API token**, use **terraform login** as the description and click on **Generate token**...



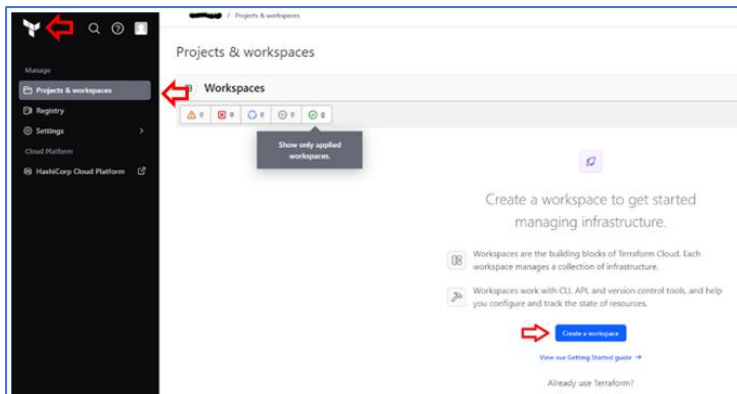
11. A new token is generated...



12. Create a local text file and paste the token in for later use.
13. We can now create our first workspace. Select 'Workspaces'...



14. Select **Create a workspace**...



15. Our workspace will be used to generate **CLI-driven workflow**...

Create a new Workspace

Workspaces determine how Terraform Cloud organizes infrastructure. A workspace contains your Terraform configuration (infrastructure as code), shared variable values, your current and historical Terraform state, and run logs. [Learn more](#) about workspaces in Terraform Cloud.

1 Choose Type 2 Connect to VCS 3 Choose a repository 4 Configure settings

Choose your workflow

Version control workflow Most common

Store your Terraform configuration in a git repository, and trigger runs based on pull requests and merges.

[Learn More](#)

CLI-driven workflow

Trigger remote Terraform runs from your local command line.

[Learn More](#)

API-driven workflow

A more advanced option. Integrate Terraform into a larger pipeline using the Terraform API.

[Learn More](#)

16. Name the new workspace **my-first-workspace**...

Create a new Workspace

Workspaces determine how Terraform Cloud organizes infrastructure. A workspace contains your Terraform configuration (infrastructure as code), shared variable values, your current and historical Terraform state, and run logs. [Learn more](#) about workspaces in Terraform Cloud.

1 Choose Type 2 Configure settings

Name Workspace

Workspace Name

my-first-workspace

The name of your workspace is unique and used in tool, tooling, and CI-CSDS. underscores, and alphanumeric characters are permitted. [Learn more about naming workspaces](#).

Project

Default Project

Description

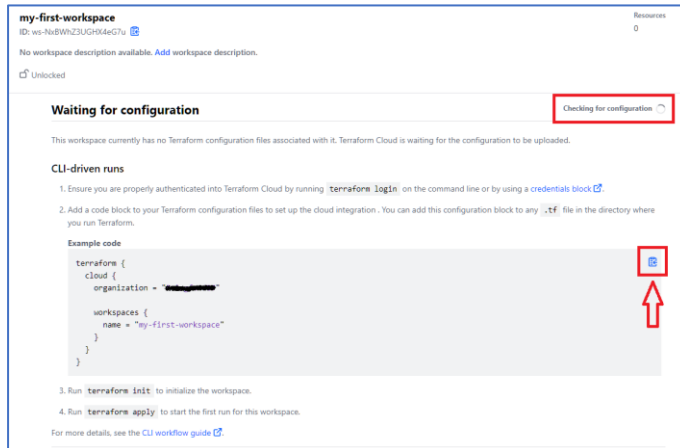
Optional

Workspace description

Create workspace Cancel

Task3 Integrate Terraform client and Terraform Cloud Workspace

1. The workspace now exists but has no configuration files associated with it. To associate our local terraform files with Terraform Cloud we need to add the command block generated in the workspace to our local configuration. Click on the copy icon..



2. Paste it into **main.tf** after the azure provider block and save the changes..

```
terraform {
  cloud {
    organization = "Your Organization Here"

    workspaces {
      name = "my-first-workspace"
    }
  }
}
```

3. We need to declare the variables used for our TFC workspace within our local files but without any values. Update the provider block of main.tf with the four variables and save the file..

```
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
  client_id = var.client_id
  client_secret = var.client_secret
  tenant_id = var.tenant_id
}
```


4. Now declare these variables in the variable.tf file and save the file...

```
variable "client_id" {  
  type = string  
}  
  
variable "client_secret" {  
  type = string  
}  
  
variable "subscription_id" {  
  type = string  
}  
  
variable "tenant_id" {  
  type = string  
}
```

5. Run **terraform init**

```
PS /home/michael/lab6> terraform init  
  
Initializing Terraform Cloud...  
  
Error: Required token could not be found  
  
Run the following command to generate a token for app.terraform.io:  
    terraform login
```

6. You need credentials on the IDE to identify yourself to Terraform Cloud. This is a token generated during the terraform login process. Run **`terraform login`** and enter **`yes`**...

```
PS /home/michael/lab6> terraform login  
Terraform will request an API token for app.terraform.io using your browser.  
  
If login is successful, Terraform will store the token in plain text in  
the following file for use by subsequent commands:  
    /home/michael/.terraform.d/credentials.tfrc.json  
  
Do you want to proceed?  
    Only 'yes' will be accepted to confirm.  
  
Enter a value: 
```

7. You are now prompted to generate a token on Terraform Cloud via the supplied url. We already created a token earlier so paste it into

the IDE terminal. **The token value will not be displayed**, simply press Enter after pasting.

```
Enter a value: yes

-----

Open the following URL to access the tokens page for app.terraform.io:
https://app.terraform.io/app/settings/tokens?source=terraform-login

-----

Generate a token using your browser, and copy-paste it into this prompt.

Terraform will store the token in plain text in the following file
for use by subsequent commands:
/home/michael/.terraform.d/credentials.tfrc.json

Token for app.terraform.io:
Enter a value: 
```

8. If successful, the **Welcome to Terraform Cloud** banner will show...

```
-----
Welcome to Terraform Cloud!
-----
Documentation: terraform.io/docs/cloud
-----
```

9. Run **terraform init**. You will be notified that the Terraform Cloud initialization completes successfully...

```
Initializing Terraform Cloud...

Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/azurerm v3.18.0

Terraform Cloud has been successfully initialized!

You may now begin working with Terraform Cloud. Try running "terraform plan" to
see any changes that are required for your infrastructure.

If you ever set or change modules or Terraform Settings, run "terraform init"
again to reinitialize your working directory.
PS /home/michael/lab6> 
```

10. Run **terraform apply**...

```
PS C:\terraform\MCG\QATIPExpV1\labs\lab6> terraform.exe apply
Running apply in Terraform Cloud. Output will stream here. Pressing Ctrl-C
will cancel the remote apply if it's still pending. If the apply started it
will stop streaming the logs, but will not stop the apply running remotely.

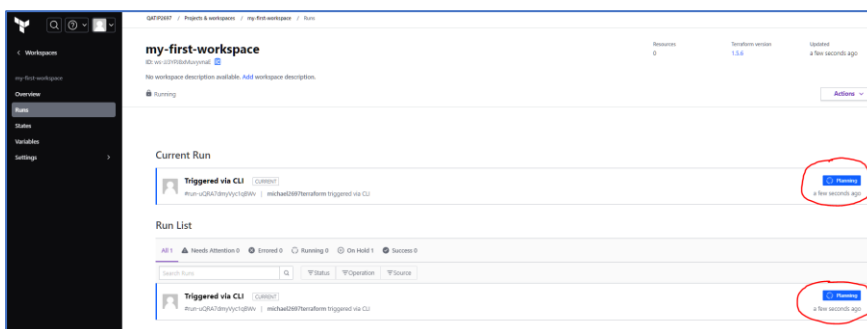
Preparing the remote apply...

To view this run in a browser, visit:
https://app.terraform.io/app/QATIP2697/my-first-workspace/runs/run-ZtgyMbk28CVxmGtv

Waiting for the plan to start...

Terraform v1.3.9
on linux_amd64
Initializing plugins and modules...
```

11. Switch to TFC page. Select 'Runs' from the main menu. Planning should begin...



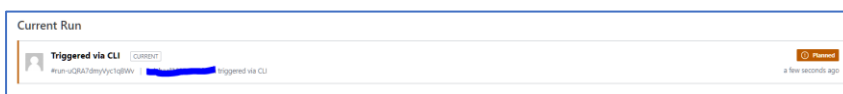
12. Switch back to IDE and wait for prompt to appear. **Do not type 'yes' yet...**

```
Plan: 5 to add, 0 to change, 0 to destroy.

Do you want to perform these actions in workspace "my-first-workspace"?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

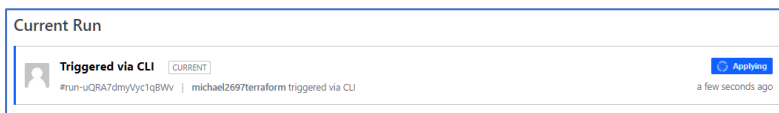
Enter a value: 
```

13. Switch to TFC page. Planning is now complete...

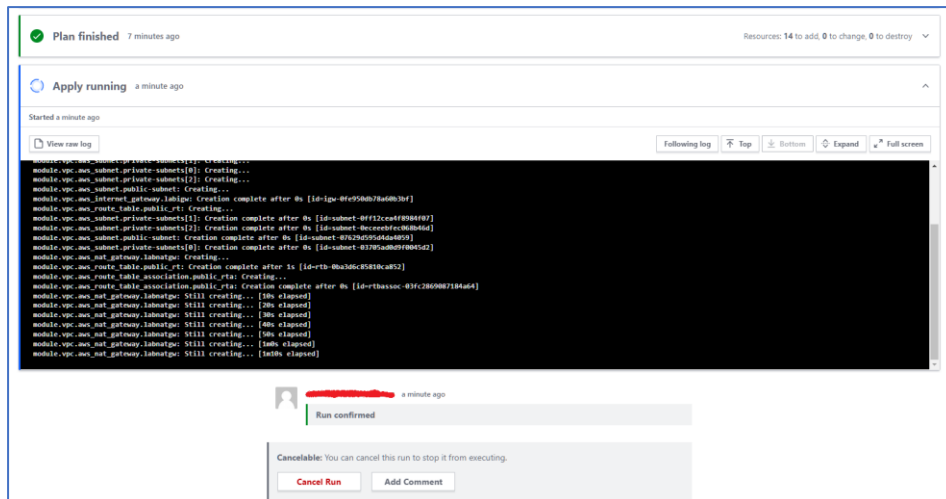


14. Switch back to IDE and enter 'yes'..

15. Switch back to TFC page. The plan is being applied..



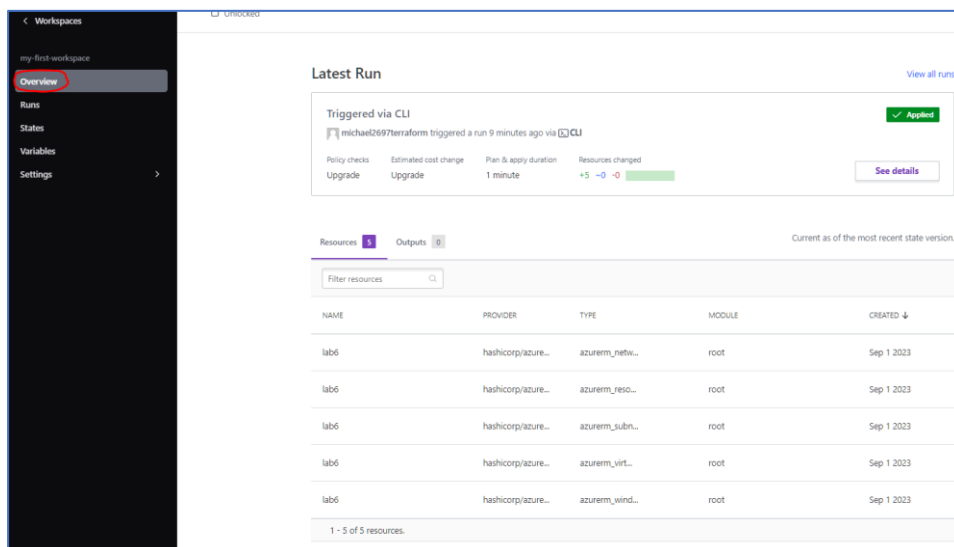
16. Click into the current run to watch deployment progressing...



17. Switch back to IDE and wait for deployment to complete...

```
azurerm_windows_virtual_machine.lab6: Creation complete after 1m14s
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
PS /home/michael/lab6>
```

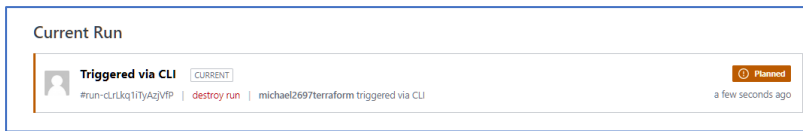
18. Switch back to TFC. Select 'Overview' to see a summary of the applied job and the resources created...



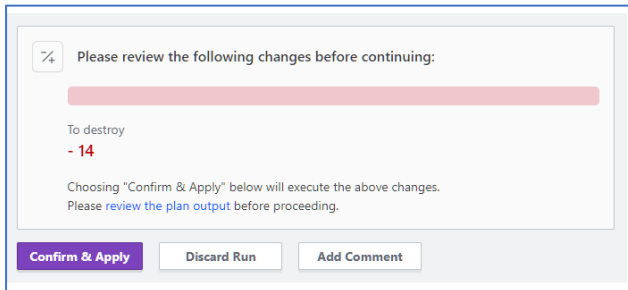
19. In the IDE enter **terraform destroy** Do not confirm yet.

20. Switch to TFC and note the planning of the destroy is in progress.

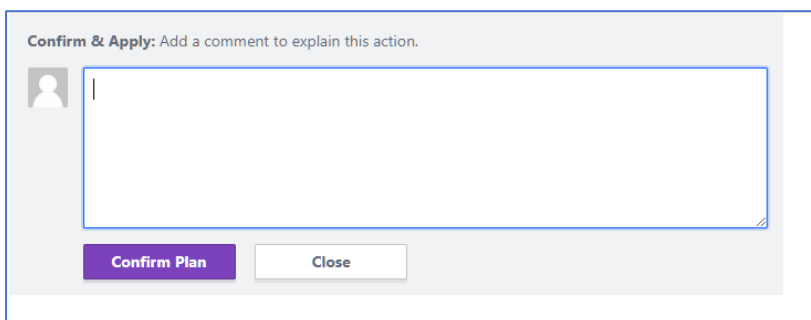
21. Wait until the destroy run shows as 'Planned' Click to see details



22. Scroll down and click on `Confirm and Apply`



23. Click on `Confirm Plan`



24. Switch back to IDE and, if timely, you will see a message indicating the apply has been approved on TFC. The destroy will then progress.

